

Assignment_2_Sql

Section A — SQL Basics (SELECT, Constraints, Primary Keys)

Q1. Write a query to display all columns and rows from the customer's table.

Ans-

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying the following SQL query:

```
SELECT *  
FROM customers;
```

The 'Result Grid' shows the output of the query, displaying all columns and rows from the 'customers' table. The columns are: customer_id, first_name, last_name, email, city, state, join_date, and is_premium. The rows are:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0

The 'Action Output' section shows the execution details: 27 17:28:24 SELECT * FROM customers LIMIT 0, 1000. The message indicates 8 row(s) returned, and the duration is 0.000 sec / 0.000 sec.

Q2. Retrieve only the first_name, last_name, and city of all customers.

Ans-

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying the following SQL query:

```
SELECT first_name,  
last_name,  
city  
FROM customers;
```

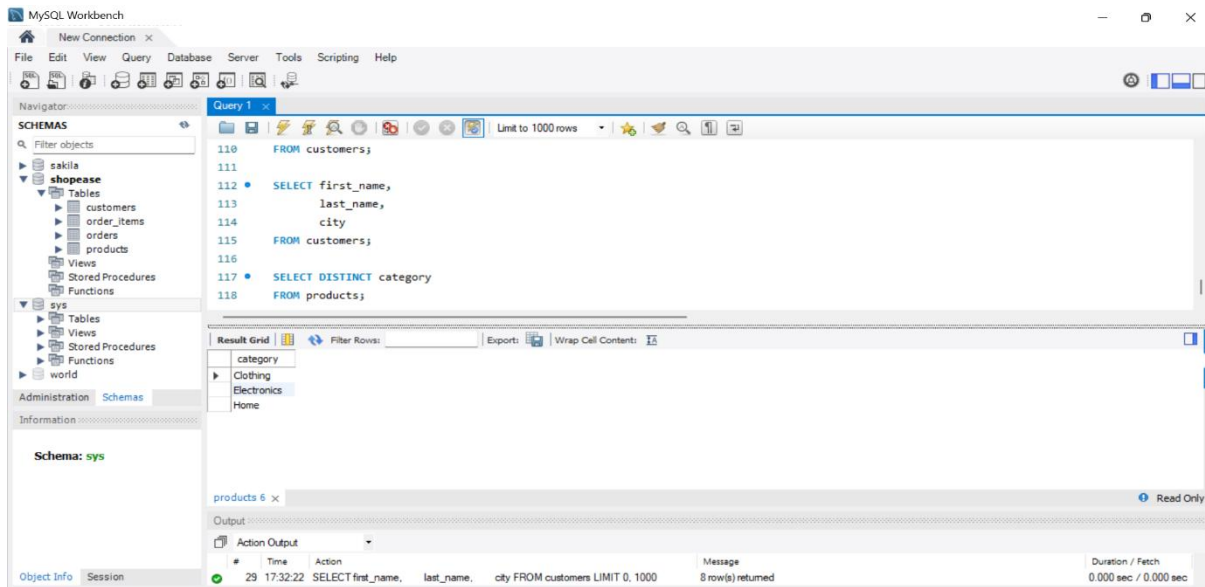
The 'Result Grid' shows the output of the query, displaying only the first_name, last_name, and city columns for all customers. The rows are:

first_name	last_name	city
Aarav	Sharma	Mumbai
Priya	Patel	Ahmedabad
Rohan	Gupta	Delhi
Sneha	Reddy	Hyderabad
Vikram	Singh	Jaipur
Ananya	Iyer	Chennai
Karan	Mehta	Pune
Divya	Nair	Kochi

The 'Action Output' section shows the execution details: 28 17:31:35 SELECT * FROM customers LIMIT 0, 1000. The message indicates 8 row(s) returned, and the duration is 0.000 sec / 0.000 sec.

Q3. List all unique categories available in the products table.

Ans-

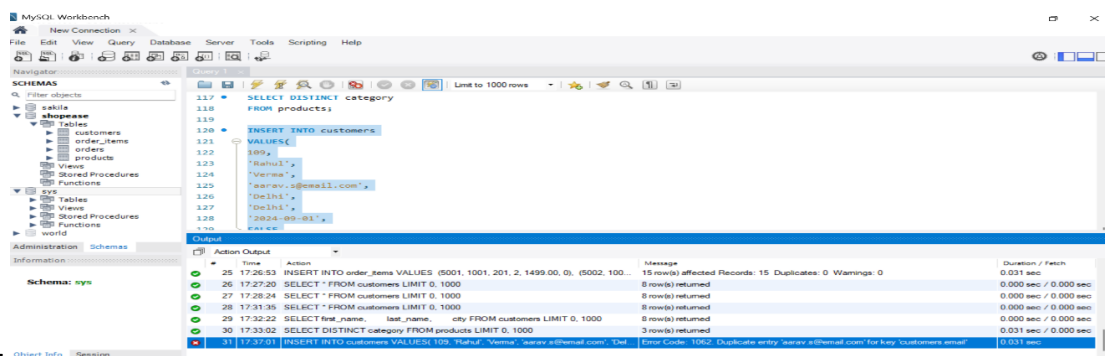


Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

Ans-

table	Primary key
Customers	Customer_id
Products	Products_id
Orders	Orders_id
Orders_item	item_id

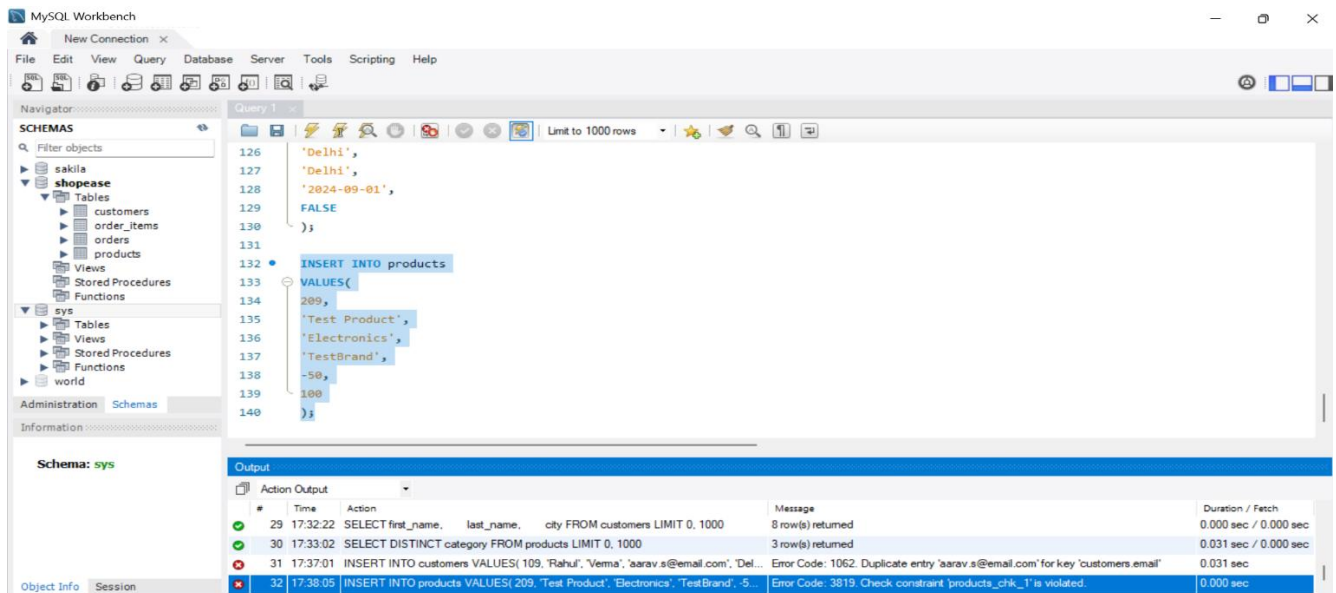
Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?



Ans-

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

Ans-



The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying the following SQL code:

```
126 'Delhi',
127 'Delhi',
128 '2024-09-01',
129 FALSE
130 );
131
132 INSERT INTO products
133 VALUES(
134 209,
135 'Test Product',
136 'Electronics',
137 'TestBrand',
138 -50,
139 100
140 );
```

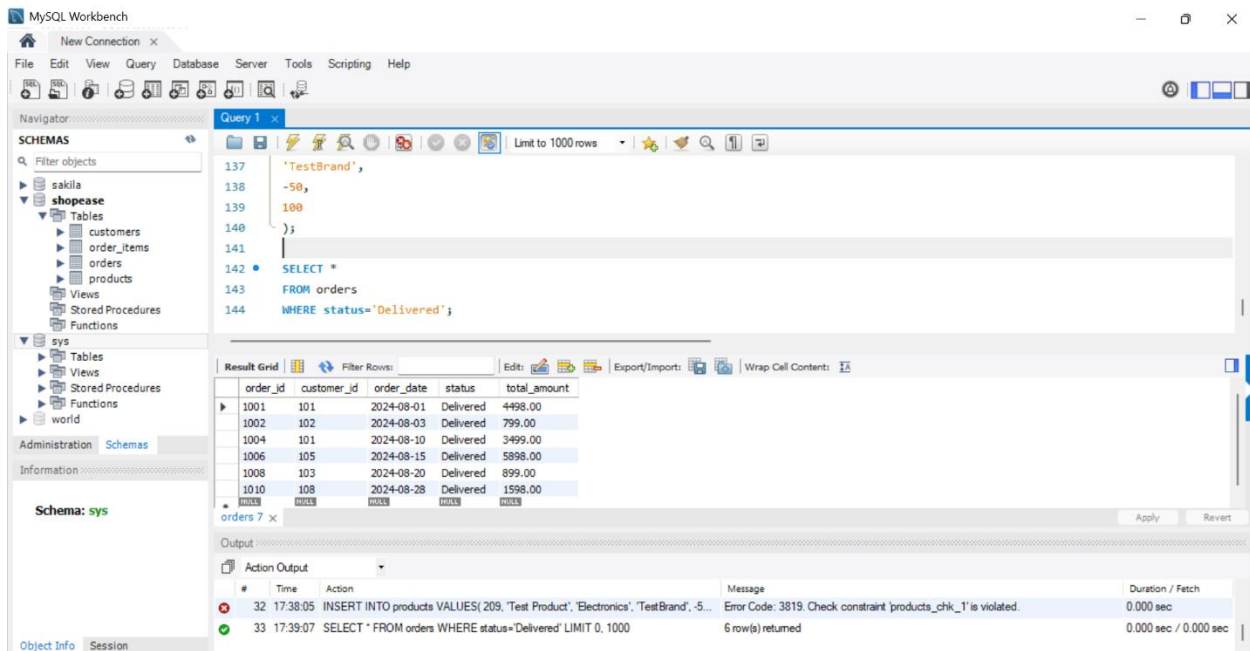
The 'Output' tab shows the execution results:

#	Time	Action	Message	Duration / Fetch
29	17:32:22	SELECT first_name, last_name, city FROM customers LIMIT 0, 1000	8 row(s) returned	0.000 sec / 0.000 sec
30	17:33:02	SELECT DISTINCT category FROM products LIMIT 0, 1000	3 row(s) returned	0.031 sec / 0.000 sec
31	17:37:01	INSERT INTO customers VALUES(109, 'Rahul', 'Vema', 'aarav.s@email.com', 'Del...	Error Code: 1062. Duplicate entry 'aarav.s@email.com' for key 'customers.email'	0.031 sec
32	17:38:05	INSERT INTO products VALUES(209, 'Test Product', 'Electronics', 'TestBrand', -5...	Error Code: 3819. Check constraint 'products_chk_1' is violated.	0.000 sec

Section B — Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'.

Ans-



The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying the following SQL code:

```
137 'TestBrand',
138 -50,
139 100
140 );
141
142 SELECT *
143 FROM orders
144 WHERE status='Delivered';
```

The 'Result Grid' shows the execution results:

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4408.00
1002	102	2024-08-03	Delivered	799.00
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1008	103	2024-08-20	Delivered	899.00
1010	108	2024-08-28	Delivered	1598.00

The 'Output' tab shows the execution results:

#	Time	Action	Message	Duration / Fetch
32	17:38:05	INSERT INTO products VALUES(209, 'Test Product', 'Electronics', 'TestBrand', -5...	Error Code: 3819. Check constraint 'products_chk_1' is violated.	0.000 sec
33	17:39:07	SELECT * FROM orders WHERE status='Delivered' LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

MySQL Workbench interface showing a query to find products in the 'Electronics' category with a unit price greater than ₹2000.

Query 1:

```

142 SELECT *
143 FROM orders
144 WHERE status='Delivered';
145
146 SELECT *
147 FROM products
148 WHERE category='Electronics'
149 AND unit_price > 2000;

```

Result Grid:

product_id	product_name	category	brand	unit_price	stock_qty
203	Smart Watch	Electronics	Noise	2999.00	150
205	Bluetooth Speaker	Electronics	JBL	3499.00	200

Output:

#	Time	Action	Message	Duration / Fetch
33	17:39:07	SELECT * FROM orders WHERE status='Delivered' LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec
34	17:39:52	SELECT * FROM products WHERE category='Electronics' AND unit_price > 2000	2 row(s) returned	0.016 sec / 0.000 sec

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

MySQL Workbench interface showing a query to list customers who joined in the year 2024 and belong to the state 'Maharashtra'.

Query 1:

```

148 WHERE category='Electronics'
149 AND unit_price > 2000;
150
151 SELECT *
152 FROM customers
153 WHERE state='Maharashtra'
154 AND join_date BETWEEN '2024-01-01'
155 AND '2024-12-31';

```

Result Grid:

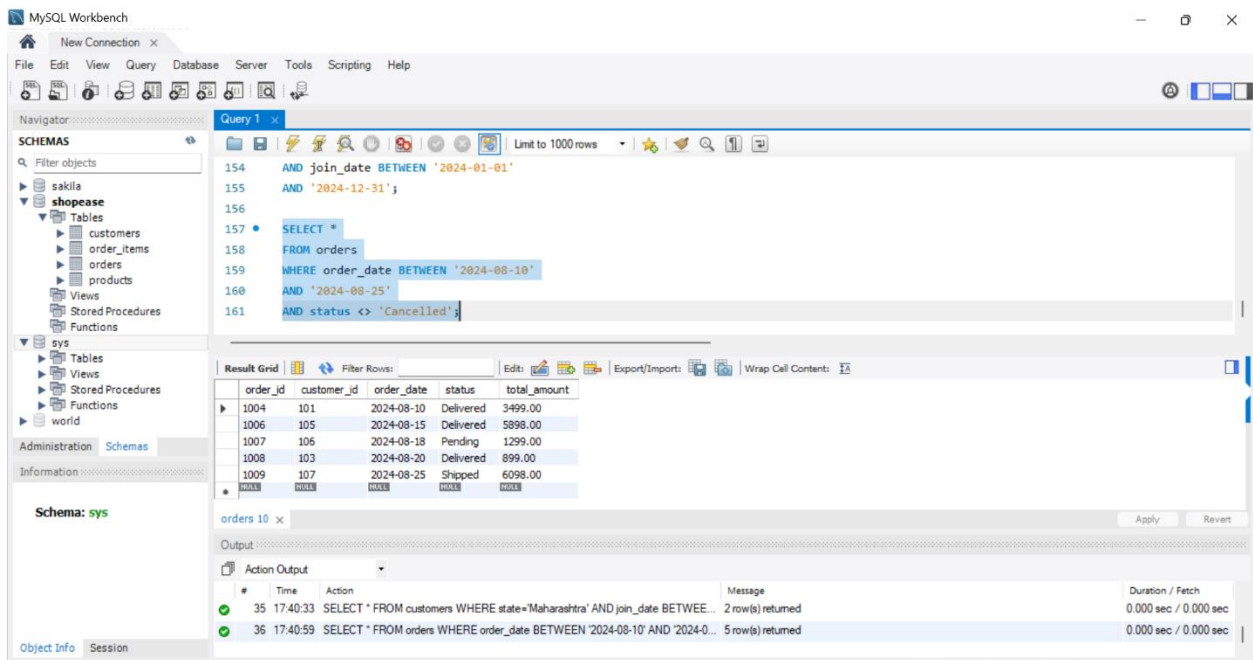
customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1

Output:

#	Time	Action	Message	Duration / Fetch
34	17:39:52	SELECT * FROM products WHERE category='Electronics' AND unit_price > 2000	2 row(s) returned	0.016 sec / 0.000 sec
35	17:40:33	SELECT * FROM customers WHERE state='Maharashtra' AND join_date BETWEEN...	2 row(s) returned	0.000 sec / 0.000 sec

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

Ans-



Q11. Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a sample query that would benefit from this index.

Ans-

CREATE INDEX idx_orders_date

ON orders(order_date);

This index speeds up searching and filtering based on order_date.

SELECT *

FROM orders

WHERE order_date BETWEEN '2024-08-01'

AND '2024-08-31';

Q12. If you run: `SELECT * FROM customers WHERE YEAR(join_date) = 2024;` — would the index on `join_date` be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

Ans- BAD QUERY

`SELECT *`

`FROM customers`

`WHERE YEAR(join_date)=2024;`

GOOD QUERY

`SELECT *`

`FROM customers`

`WHERE join_date >= '2024-01-01'`

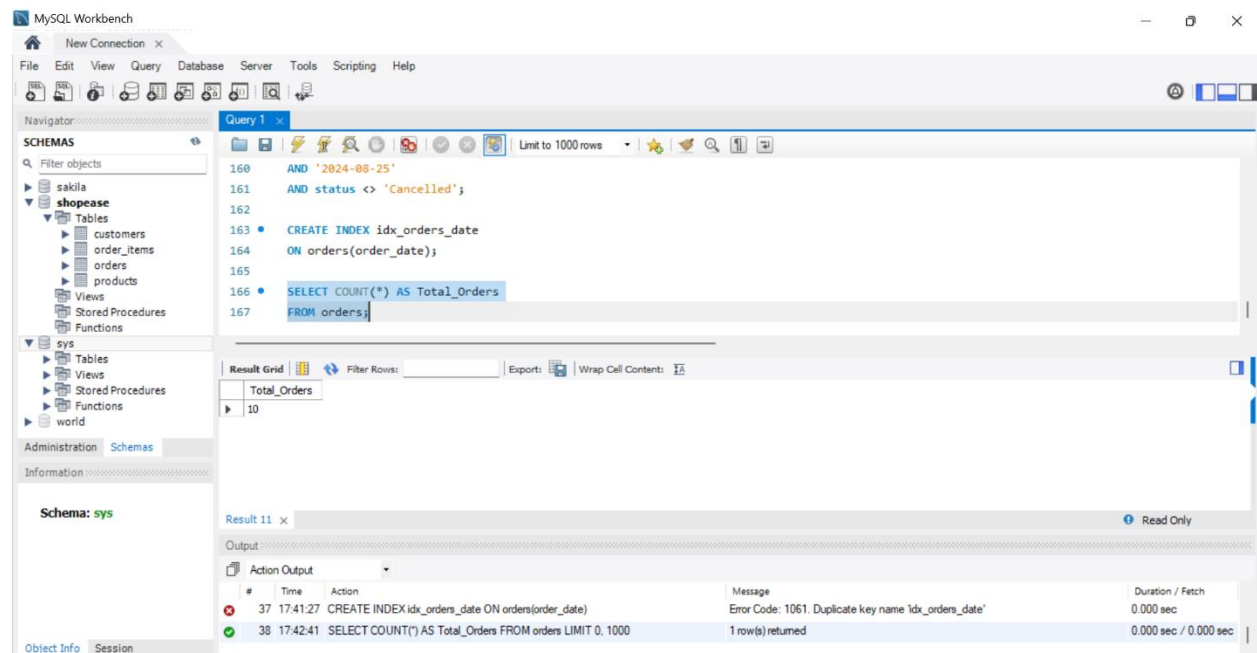
`AND join_date < '2025-01-01';`

The first query applies a function on the column, preventing efficient index usage.

Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.

Ans-



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' panel with a tree view showing databases like 'sakila' and 'shopease'. The 'shopease' database is selected, and its tables are listed. The main query editor shows the following SQL code:

```
160 AND '2024-08-25'  
161 AND status <> 'Cancelled';  
162  
163 • CREATE INDEX idx_orders_date  
164 ON orders(order_date);  
165  
166 • SELECT COUNT(*) AS Total_Orders  
167 FROM orders;
```

The 'Result Grid' shows the output of the query:

Total_Orders
10

The 'Output' panel at the bottom shows the 'Action Output' for the query execution:

#	Time	Action	Message	Duration / Fetch
37	17:41:27	CREATE INDEX idx_orders_date ON orders(order_date)	Error Code: 1061. Duplicate key name 'idx_orders_date'	0.000 sec
38	17:42:41	SELECT COUNT(*) AS Total_Orders FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

Ans-

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' panel with a tree view of the 'sakila' database, including tables like 'customers', 'order_items', 'orders', and 'products'. The main query editor shows the following SQL code:

```
164 ON orders(order_date);
165
166 • SELECT COUNT(*) AS Total_Orders
167 FROM orders;
168
169 • SELECT SUM(total_amount) AS Delivered_Revenue
170 FROM orders
171 WHERE status='Delivered';
```

The 'Result Grid' below the query editor shows the results of the second query:

Delivered_Revenue
17191.00

The 'Output' panel at the bottom shows the execution log for two queries:

#	Time	Action	Message	Duration / Fetch
38	17:42:41	SELECT COUNT(*) AS Total_Orders FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
39	17:43:10	SELECT SUM(total_amount) AS Delivered_Revenue FROM orders WHERE status='Delivered';	1 row(s) returned	0.031 sec / 0.000 sec

Q15. Calculate the average unit_price of products in each category.

Ans-

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' panel with a tree view of the 'sakila' database, including tables like 'customers', 'order_items', 'orders', and 'products'. The main query editor shows the following SQL code:

```
169 • SELECT SUM(total_amount) AS Delivered_Revenue
170 FROM orders
171 WHERE status='Delivered';
172
173 • SELECT category,
174 AVG(unit_price) AS Avg_Price
175 FROM products
176 GROUP BY category;
```

The 'Result Grid' below the query editor shows the results of the second query:

category	Avg_Price
Clothing	2699.000000
Electronics	2224.000000
Home	949.000000

The 'Output' panel at the bottom shows the execution log for two queries:

#	Time	Action	Message	Duration / Fetch
39	17:43:10	SELECT SUM(total_amount) AS Delivered_Revenue FROM orders WHERE status='Delivered';	1 row(s) returned	0.031 sec / 0.000 sec
40	17:43:46	SELECT category, AVG(unit_price) AS Avg_Price FROM products GROUP BY category;	3 row(s) returned	0.032 sec / 0.000 sec

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

Ans-

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
176 GROUP BY category;
177
178 SELECT status,
179        COUNT(*) AS Order_Count,
180        SUM(total_amount) AS Revenue
181 FROM orders
182 GROUP BY status
183 ORDER BY Revenue DESC;
```

The Result Grid displays the following data:

status	Order_Count	Revenue
Delivered	6	17191.00
Shipped	2	13596.00
Cancelled	1	2999.00
Pending	1	1299.00

The Action Output pane shows the execution details:

#	Time	Action	Message	Duration / Fetch
40	17:43:46	SELECT category, AVG(unit_price) AS Avg_Price FROM products GROUP BY...	3 row(s) returned	0.032 sec / 0.000 sec
41	17:44:09	SELECT status, COUNT(*) AS Order_Count, SUM(total_amount) AS Revenue...	4 row(s) returned	0.000 sec / 0.000 sec

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

Ans-

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
182 GROUP BY status
183 ORDER BY Revenue DESC;
184
185 SELECT category,
186        MAX(unit_price) AS Most_Expensive,
187        MIN(unit_price) AS Cheapest
188 FROM products
189 GROUP BY category;
```

The Result Grid displays the following data:

category	Most_Expensive	Cheapest
Clothing	4599.00	799.00
Electronics	3499.00	899.00
Home	1299.00	599.00

The Action Output pane shows the execution details:

#	Time	Action	Message	Duration / Fetch
41	17:44:09	SELECT status, COUNT(*) AS Order_Count, SUM(total_amount) AS Revenue...	4 row(s) returned	0.000 sec / 0.000 sec
42	17:45:04	SELECT category, MAX(unit_price) AS Most_Expensive, MIN(unit_price) AS Cheapest...	3 row(s) returned	0.000 sec / 0.000 sec

Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)

Ans-

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
188 FROM products
189 GROUP BY category;
190
191 SELECT category,
192        AVG(unit_price) AS Avg_Price
193 FROM products
194 GROUP BY category
195 HAVING AVG(unit_price) > 2000;
```

The Result Grid shows the following data:

category	Avg_Price
Clothing	2699.000000
Electronics	2224.000000

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
42	17:45:04	SELECT category, MAX(unit_price) AS Most_Expensive, MIN(unit_price) AS	3 row(s) returned	0.000 sec / 0.000 sec
43	17:45:51	SELECT category, AVG(unit_price) AS Avg_Price FROM products GROUP BY	2 row(s) returned	0.000 sec / 0.000 sec

Section D — Joins & Relationships

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name.

Show: order_id, order_date, first_name, last_name, total_amount.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
197 SELECT o.order_id,
198        o.order_date,
199        c.first_name,
200        c.last_name,
201        o.total_amount
202 FROM orders o
203 INNER JOIN customers c
204 ON o.customer_id = c.customer_id;
```

The Result Grid shows the following data:

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498.00
1004	2024-08-10	Aarav	Sharma	3499.00
1002	2024-08-03	Priya	Patel	799.00
1003	2024-08-05	Rohan	Gupta	7498.00
1008	2024-08-20	Rohan	Gupta	899.00
1005	2024-08-12	Sneha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5998.00
1007	2024-08-18	Ananya	Iyer	1299.00
1009	2024-08-25	Karan	Mehra	6998.00
1010	2024-08-28	Divya	Nair	1598.00

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
43	17:45:51	SELECT category, AVG(unit_price) AS Avg_Price FROM products GROUP BY	2 row(s) returned	0.000 sec / 0.000 sec

Ans-

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

Ans-

MySQL Workbench interface showing a query for Q20. The query is as follows:

```

206 SELECT c.customer_id,
207        c.first_name,
208        c.last_name,
209        o.order_id,
210        o.order_date
211 FROM customers c
212 LEFT JOIN orders o
213 ON c.customer_id = o.customer_id;

```

The result grid displays the following data:

customer_id	first_name	last_name	order_id	order_date
101	Aarav	Sharma	1001	2024-08-01
101	Aarav	Sharma	1004	2024-08-10
102	Priya	Patel	1002	2024-08-03
103	Rohan	Gupta	1003	2024-08-05
103	Rohan	Gupta	1008	2024-08-20
104	Sneha	Reddy	1005	2024-08-12
105	Vikram	Singh	1006	2024-08-15
106	Ananya	Iyer	1007	2024-08-18
107	Karan	Mehta	1009	2024-08-25
108	Divya	Nair	1010	2024-08-28

Q21. Write a query using JOINS across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

Ans-

MySQL Workbench interface showing a query for Q21. The query is as follows:

```

214
215 SELECT o.order_id,
216        p.product_name,
217        oi.quantity,
218        oi.unit_price,
219        oi.discount_pct
220 FROM orders o
221 JOIN order_items oi
222 ON o.order_id = oi.order_id

```

The result grid displays the following data:

order_id	product_name	quantity	unit_price	discount_pct
1003	Running Shoes	1	4599.00	5.00
1006	Running Shoes	1	4599.00	5.00
1004	Bluetooth Speaker	1	3499.00	0.00
1009	Bluetooth Speaker	1	3499.00	0.00
1007	Bedsheet Set	1	1299.00	0.00
1010	Bedsheet Set	1	1299.00	0.00
1001	Laptop Stand	1	899.00	10.00
1008	Laptop Stand	1	899.00	0.00
1009	Cushion Covers (...)	2	599.00	15.00

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

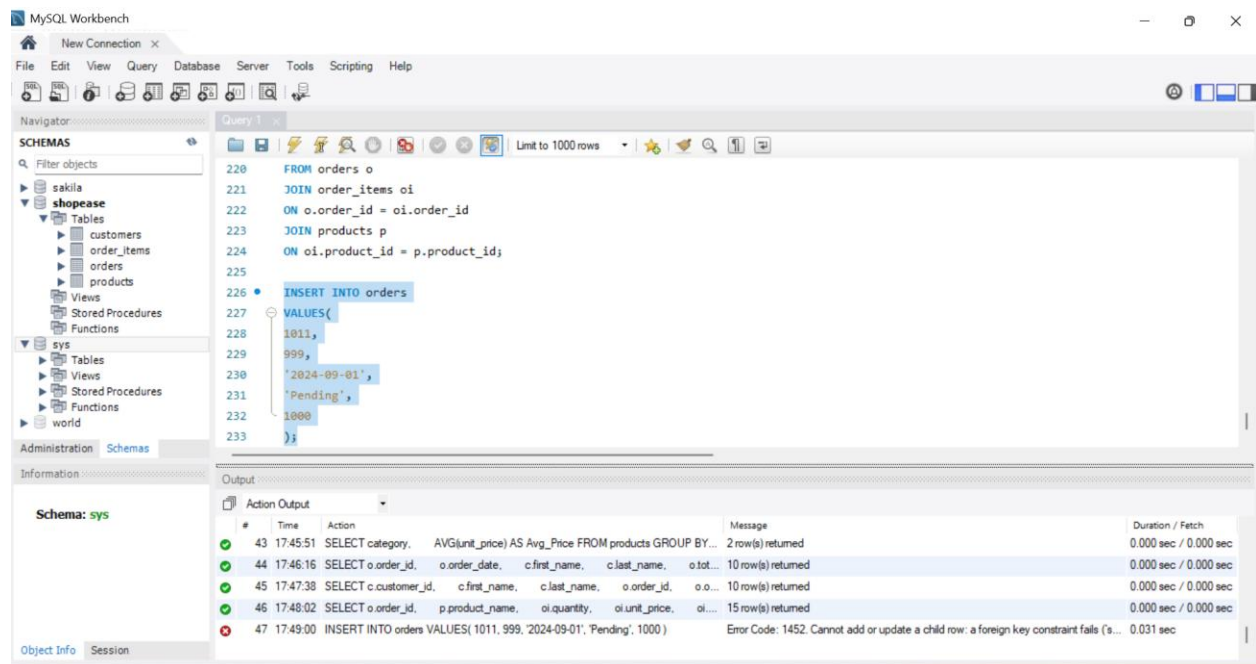
Ans- LEFT JOIN returns all records from left table.

RIGHT JOIN returns all records from right table.

FULL OUTER JOIN returns all records from both tables.

Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

Ans-



The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane displays the database structure, including tables like customers, order_items, orders, and products. The main query editor shows a query that joins orders, order_items, and products tables, followed by an INSERT statement into the orders table with values (1011, 999, '2024-09-01', 'Pending', 1000). The 'Output' pane at the bottom shows the execution results, including a message indicating a foreign key constraint failure (Error Code: 1452) because the customer_id 999 does not exist in the customers table.

```
220 FROM orders o
221 JOIN order_items oi
222 ON o.order_id = oi.order_id
223 JOIN products p
224 ON oi.product_id = p.product_id;
225
226 INSERT INTO orders
227 VALUES(
228 1011,
229 999,
230 '2024-09-01',
231 'Pending',
232 1000
233 );
```

#	Time	Action	Message	Duration / Fetch
43	17:45:51	SELECT category, AVG(unit_price) AS Avg_Price FROM products GROUP BY...	2 row(s) returned	0.000 sec / 0.000 sec
44	17:46:16	SELECT o.order_id, o.order_date, c.first_name, c.last_name, o.tot...	10 row(s) returned	0.000 sec / 0.000 sec
45	17:47:38	SELECT c.customer_id, c.first_name, c.last_name, o.order_id, o.o...	10 row(s) returned	0.000 sec / 0.000 sec
46	17:48:02	SELECT o.order_id, p.product_name, oi.quantity, oi.unit_price, oi...	15 row(s) returned	0.000 sec / 0.000 sec
47	17:49:00	INSERT INTO orders VALUES(1011, 999, '2024-09-01', 'Pending', 1000)	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('s...	0.031 sec

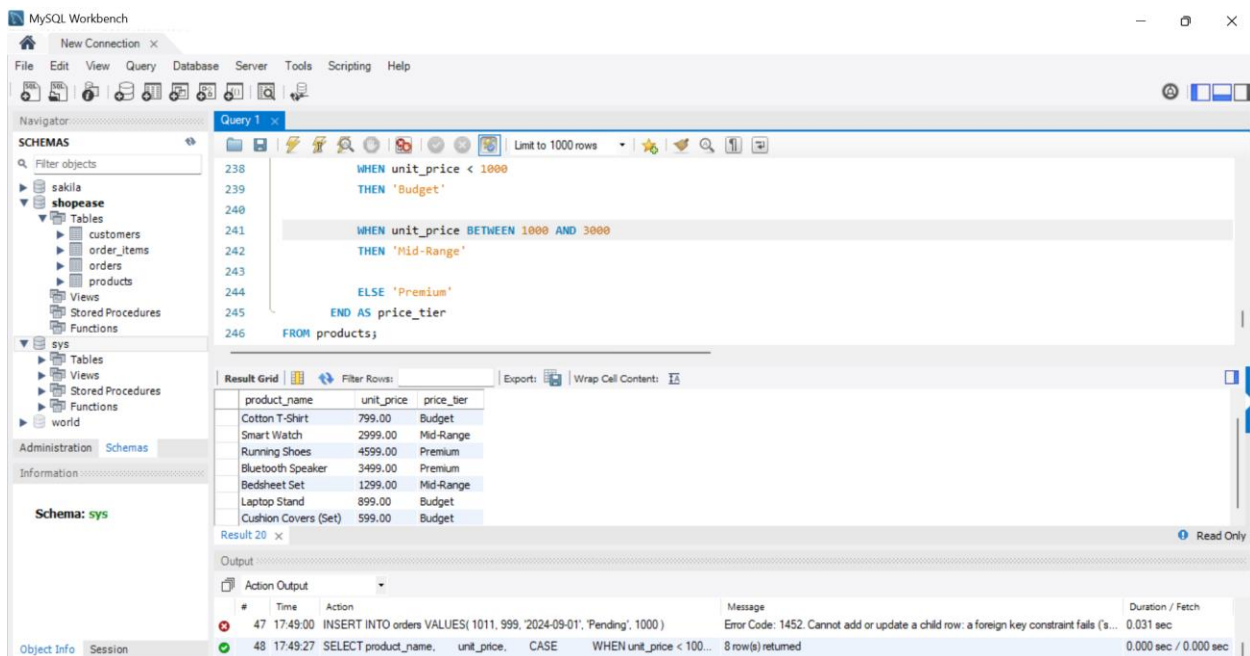
Section E — Advanced Concepts (CASE, ACID, Transactions)

Q24. Write a query using CASE to classify products into price tiers:

- 'Budget' → unit_price < 1000
- 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
- 'Premium' → unit_price > 3000

Display: product_name, unit_price, price_tier.

Ans-



Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

Q26. Explain each letter of ACID:

- A – Atomicity
- C – Consistency
- I – Isolation
- D – Durability

Give a real-world example (e.g., bank transfer) showing why each property is important.

Ans- **Atomicity**-All operations complete or none complete.

Consistency-Database remains valid.

Isolation-Transactions do not interfere.

Durability-Committed data remains permanent.

Example: Bank transfer.

Q27. Write a SQL transaction that does the following atomically:

1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)
2. Insert two order items for that order

3. Update the stock_qty of the purchased products
 4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.
- Write the complete BEGIN...COMMIT/ROLLBACK block.

Ans-

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'sakila' and 'shopease' databases. The 'shopease' database is expanded, showing tables like 'customers', 'order_items', 'orders', and 'products'. The main editor window shows a SQL query with the following lines:

```
279 VALUES(  
280 5017,  
281 1011,  
282 208,  
283 1,  
284 599,  
285 0  
286 );  
287  
288 • UPDATE products  
289 SET stock_qty=stock_qty-1  
290 WHERE product_id=202;  
291  
292 • UPDATE products  
293 SET stock_qty=stock_qty-1  
294 WHERE product_id=208;  
295  
296 • COMMIT;
```

The 'Output' tab at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
55	17:51:28	UPDATE products SET stock_qty=stock_qty-1 WHERE product_id=208	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
56	17:51:28	COMMIT	0 row(s) affected	0.000 sec